Śmieci kosmiczne według właściciela

Kontekst

Fikcyjna firma **Kosmo Porządek** korzysta ze skryptu `main.py`, który wczytuje `data/satcat.csv` i pokazuje liczbę oraz podgląd rekordów. Firma zleca rozbudowę skryptu, ponieważ analitycy chcą policzyć wpisy o śmieciach kosmicznych dla wybranego kodu właściciela.

Podział pracy

Student 1 - analiza w pandas

1. **Raport podstawowy.** Uzupełnij `analyze`. Program ma policzyć rekordy, dla których OBJECT_TYPE ma wartość DEB, a OWNER odpowiada podanemu kodowi.
2. **Podsumowanie.** Uzupełnij `summarize`. Zestaw śmieci kosmiczne wybranego właściciela według dekady startu. Pomiń brakujące daty. Dekada = 10 × floor(rok / 10). Policz rekordy w każdej dekadzie.

W obu funkcjach analizy zastosuj dodatkowe filtry `--from-year` i `--to-year`, jeśli użytkownik je poda.

Student 2 - polecenia CLI w argparse

1. **Polecenie `report`.** Dodaj podpolecenie z kryterium z przykładu i opcją `--output`. Wywołaj analizę i zapis CSV. Sprawdź poprawność argumentów.
2. **Polecenie `summary`.** Dodaj podpolecenie z tym samym kryterium i opcjonalnym `--plot`. Wypisz podsumowanie w konsoli. Gdy podano ścieżkę PNG, zapisz również wykres. Zachowaj `--data` i podgląd startera.

W obu podpoleceniach dodaj do `argparse` opcjonalne filtry `--from-year` i `--to-year`. Sprawdź ich wartości i przekaż je do funkcji analizy.

Student 3 - pliki wynikowe

1. **Eksport CSV.** Uzupełnij `write_report`. Użyj UTF-8 i nagłówka `OWNER,COUNT`. Przy pustej tabeli zapisz sam nagłówek.
2. **Wykres PNG.** Uzupełnij `write_plot` w Matplotlib. Na osi X pokaż dekada startu, a na osi Y liczbę rekordów lub startów w danej kategorii. Podpisz obie osie i zapisz wykres pod wskazaną ścieżką.

Wpiszcie nazwiska i polecenia do README. Każdy wnosi obie funkcjonalności na własnej gałęzi. Przed scaleniem ktoś z zespołu przegląda zmianę. Skonfigurujcie Poetry według instrukcji na następnej stronie.

Oczekiwany rezultat i zaliczenie

Prowadzący klonuje oddane repozytorium i uruchamia poniższe polecenia. Sprawdźcie, czy wynik zawiera jeden wiersz z kodem i liczbą śmieci kosmicznych, także gdy wynosi ona zero oraz czy wysokości słupków odpowiadają licznikom w podsumowaniu.

```
poetry install
poetry run python main.py --help
poetry run python main.py report --owner CIS --output wynik.csv
poetry run python main.py summary --owner CIS --plot wykres.png
```

Oddajcie adres repozytorium z działającą wersją na gałęzi głównej, historią wkładu zespołu i plikiem danych.

Szczegóły wykonania

Student 1 - dane i reguły analizy

- `analyze(records, criterion, filters=None)` i `summarize(records, criterion, filters=None)` dostają listę słowników z `read_catalog`, kryterium z CLI i opcjonalne filtry. Obie zwracają osobne tabele pandas. Raport ma kolumny `OWNER,COUNT`, a podsumowanie kolumny podane niżej.
- **Walidacja i wybór.** `--owner` przyjmuje niepusty kod. Usuń otaczające spacje i zamień kod na wielkie litery. Wybierz rekordy z OBJECT_TYPE równym DEB i OWNER równym temu kodowi. W obu funkcjach stosuj wszystkie aktywne filtry.
- **Zawartość wyniku.** W raporcie policz wszystkie wybrane rekordy, również bez daty startu. W podsumowaniu pomiń wiersze bez poprawnej daty ISO YYYY-MM-DD. Wypisz `DECADE,COUNT` w kolejności rosnących dekad.

Student 2 - zachowanie poleceń

- `report` wymaga kryterium i `--output SCIEZKA`. Wywołaj `analyze`, przekaż tabelę do `write_report`, a po zapisie wypisz ścieżkę do CSV.
- `summary` wymaga tego samego kryterium. Wypisz nagłówek i wiersze podsumowania. Jeśli podano `--plot SCIEZKA`, przekaż wynik do `write_plot`. Bez tej opcji polecenie wypisuje tylko tekst.
- **Dane i błędy.** Opcję globalną `--data SCIEZKA` podaje się przed podpoleceniem. Bez niej użyj pliku z katalogu `data/`. Błędny argument, brak pliku wejściowego lub błąd zapisu ma dać komunikat na stderr i niezerowy kod wyjścia.
- **Filtry dodatkowe.** Dodaj obie opcje w `report` i `summary`, sprawdź argumenty według następnej strony i przekaż słownik `filters` do funkcji studenta 1. Użyj kluczy argparse, np. `min_period` dla `--min-period`. Brak opcji przekaż jako `None`.

Student 3 - format plików

- **CSV.** Zapisz kolumny w kolejności z pierwszej strony, z przecinkiem jako separatorem i bez indeksu pandas. Użyj ścieżki przekazanej do funkcji. Zwróć sterowanie do CLI po zapisaniu pliku.
- **PNG.** Narysuj wykres słupkowy w kolejności z podsumowania. Przy pustej tabeli zapisz wykres z podpisami osi i napisem „Brak danych”. Po zapisie zamknij figurę.

Konfiguracja Poetry

Skonfigurujcie projekt w dostarczonym `pyproject.toml`. Uzupełnijcie opis i autorów oraz sprawdźcie zakres wersji Pythona.

W terminalu, w katalogu zawierającym `pyproject.toml`, dodajcie wymagane biblioteki:

```
poetry add pandas matplotlib
```

Aby dodać kolejny pakiet, wpiszcie `poetry add NAZWA_PAKIETU`, np. `poetry add numpy`, jeśli używacie NumPy bezpośrednio w kodzie. Polecenie instaluje pakiet i aktualizuje `pyproject.toml` oraz `poetry.lock`. Zapiszcie oba pliki w Git. Po sklonowaniu repozytorium zależności instaluje się poleceniem `poetry install`.

Zakres konfiguracji znajdziecie w dokumentacji https://python-poetry.org/docs/pyproject/ oraz w wykładzie dostępnym na Classroomie.

Przykładowe uruchomienie

Opcje main.py

`--data` wskazuje CSV przed podpoleceniem. Oba podpolecenia przyjmują kryterium `--owner` i poniższe filtry. `report` wymaga `--output`, a `summary` zapisuje wykres po podaniu `--plot`.

`--from-year` i `--to-year` przyjmują czterocyfrowe lata startu. Obie granice są włączne. Przy dowolnej aktywnej granicy pomiń rekord bez poprawnej daty startu w obu poleceniach. Początek późniejszy niż koniec to błąd. Dopuszczalne lata to 1000-9999. Przy aktywnym progu kąta pomiń wiersz z błędnym kątem, chyba że zadanie wymaga błędu dla wskazanego ID.

Wklej całe polecenie do PowerShella w katalogu repozytorium. Znak na końcu pierwszego wiersza kontynuuje polecenie. Wielokropek oznacza dalsze wiersze wyniku.

Raport CSV

```
poetry run python main.py --data data/satcat.csv report --owner CIS `
    --from-year 1960 --to-year 2010 --output wynik.csv
```

Konsola wypisuje ścieżkę `wynik.csv`. Kolejność kolumn w pliku to `OWNER,COUNT`. Jeden wiersz z kodem właściciela i łączną liczbą przypisanych mu śmieci kosmicznych. W tym uruchomieniu plik ma **1 wierszy danych** oraz nagłówek.

```
OWNER,COUNT
CIS,16461
```

Podsumowanie i wykres

```
poetry run python main.py --data data/satcat.csv summary --owner CIS `
    --from-year 1960 --to-year 2010 --plot wykres.png
```

Kolejność kolumn w konsoli to `DECADE,COUNT`. Jeden wiersz na dekadę z poprawną datą co najmniej jednego startu, rosnąco według dekady. Oczekiwany tekst:

```
DECADE,COUNT
1960,1305
1970,3300
1980,7134
1990,3365
2000,1263
...
```

Układ wykresu PNG

Program zapisuje pionowy wykres słupkowy w `wykres.png`.

- **Tytuł.** Śmieci kosmiczne według dekady startu
- **Oś X.** „Dekada startu”. Etykiety pochodzą z pierwszej kolumny podsumowania i zachowują kolejność jego wierszy.
- **Oś Y.** „Liczba śmieci kosmicznych”. Wysokości słupków pochodzą z drugiej kolumny podsumowania. Skala zaczyna się od zera.